

دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه دهم: لیست پیوندی

دکتر شهرام گلزاری

پاییز ۱۴۰۵

اهداف جلسه

در پایان این جلسه انتظار داریم که:

- اجرای LIST-SEARCH را روی یک لیست دنبال کنیم و توضیح دهیم چرا در بدترین حالت $\Theta(n)$ است.
- توضیح دهیم چرا با اشاره‌گر $L.tail$ درج در انتها $O(1)$ می‌شود.
- تغییر اشاره‌گرها را در LIST-PREPEND، LIST-INSERT و LIST-DELETE روی لیست دوسویه گام‌به‌گام دنبال کنیم.
- بتوانیم هزینه عملیات‌های مختلف در آرایه، لیست پیوندی یک‌سویه و لیست پیوندی دوسویه را با هم مقایسه کنیم.
- بتوانیم اعمال پشته و صف را توصیف کنیم.

ادامه‌ی جلسه‌ی قبل: اعمال لیست یک‌سویه

رویه‌های اعمال اصلی (۲)

```
DELETE-FIRST(L)
1  if isEMPTY(L)
2     then error list is empty
3  n = FIRST(L)
4  L.head = n.next
5  FREE-NODE(n)
6  L.size = L.size − 1
```

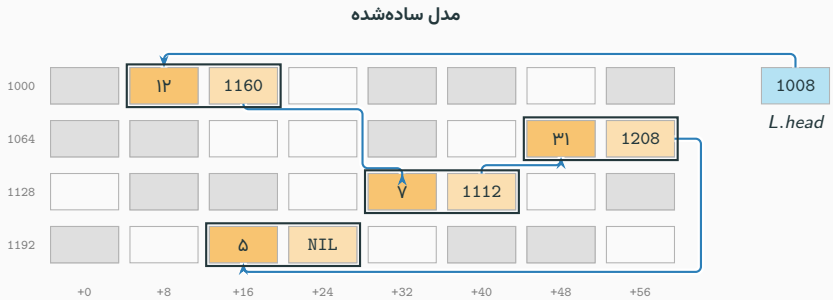
```
DELETE-AFTER(L, n)
1  if isEMPTY(L) or n == NIL or n.next == NIL
2     then error element does not exist
3  r = n.next
4  n.next = r.next
5  FREE-NODE(r)
6  L.size = L.size − 1
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۵، با نمادگذاری *CLRS*

- خط ۴ رویه‌ی DELETE-AFTER گره r را از زنجیره دور می‌زند: n مستقیم به گره پس از r اشاره می‌کند.
- خط ۳ لازم است: بدون r، پس از خط ۴ دیگر نشانی گره حذف‌شده را نداریم تا آن را با FREE-NODE آزاد کنیم.
- ترتیب دستورها مهم است، چون تغییر یک اشاره‌گر ممکن است تنها راه رسیدن به یک گره را از بین ببرد.

INSERT-AFTER در حافظه: درج ۸ پس از گره ۷

$n.next = \text{ALLOCATE-NODE}(x, n.next)$ با $x = ۸$ و n گره ۷ (نشانی ۱۱۶۰)



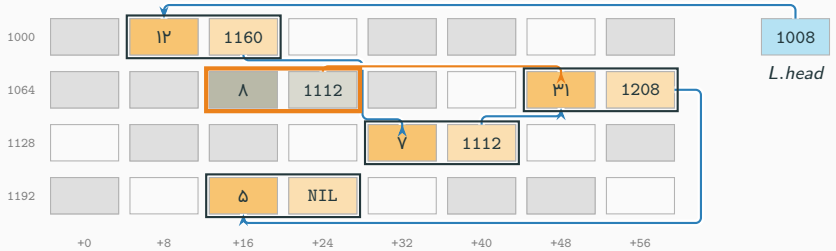
پیش از درج: $n.next = ۱۱۱۲$، یعنی گره ۷ به گره ۳۱ اشاره می‌کند.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۴-۲ ب)

INSERT-AFTER در حافظه: درج ۸ پس از گره ۷

$n.next = \text{ALLOCATE-NODE}(x, n.next)$ با $x = 8$ و n گره ۷ (نشانی ۱۱۶۰)

مدل ساده‌شده

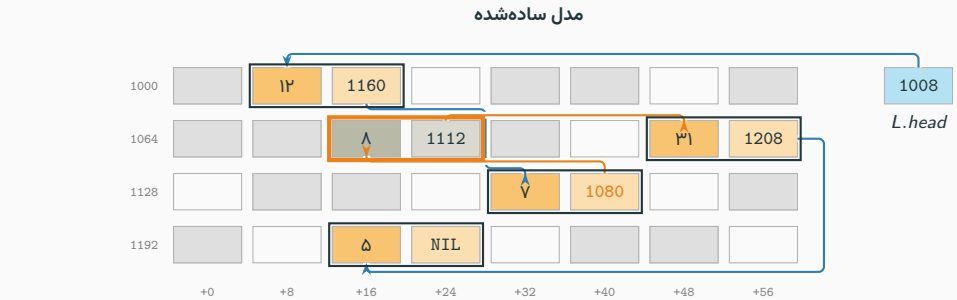


ALLOCATE-NODE(8, 1112) گره تازه را در یک بلوک آزاد دلخواه (دو خانه‌ی آزاد کنار هم)، این‌جا نشانی ۱۰۸۰، می‌سازد. مؤلفه‌ی next آن مقدار قبلی $n.next$، یعنی ۱۱۱۲، را می‌گیرد.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۴-۲ ب)

INSERT-AFTER در حافظه: درج ۸ پس از گره ۷

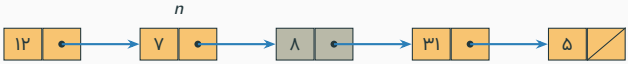
$n.next = \text{ALLOCATE-NODE}(x, n.next)$ با $x = ۸$ و n گره ۷ (نشانی ۱۱۶۰)



نشانی گره تازه در $n.next$ نوشته می‌شود. ترتیب منطقی اکنون $\langle ۱۲, ۷, ۸, ۳۱, ۵ \rangle$ است و هیچ عنصری جابه‌جا نشد: فقط دو خانه‌ی next مقدار گرفتند.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۴-۲ ب)

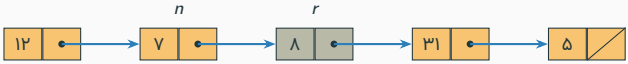
DELETE-AFTER: حذف گره پس از n



می‌خواهیم گره پس از n (گره ۸) را حذف کنیم.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۴-۲ ت)

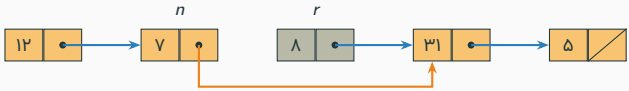
DELETE-AFTER: حذف گره پس از n



خط ۳: $r = n.next$. نشانی گره ۸ را در r نگه می‌داریم.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۴-۲ ت)

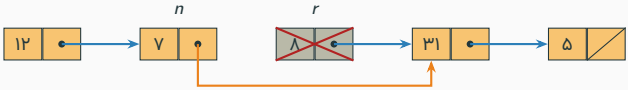
DELETE-AFTER: حذف گره پس از n



خط ۴: $n.next = r.next$. اکنون n مستقیم به گره ۳۱ اشاره می‌کند و گره ۸ از زنجیره بیرون افتاده است.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۴-۲ ت)

DELETE-AFTER: حذف گره پس از n



خط ۵: Free-Node(r). حافظه‌ی گره ۸ آزاد می‌شود. باز هم هیچ عنصری جابه‌جا نشد.

بر اساس داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۶ (شکل ۴-۲ ت)

پیمایش و جست‌وجو

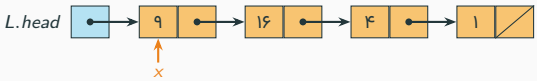
جست‌وجو در لیست: List-Search

List-Search(L, k)

1 $x = L.head$
2 **while** $x \neq$ NIL **and** $x.key \neq k$
3 $\quad x = x.next$
4 **return** x

CLRS, 4th ed.، صفحه ۲۶۰

مثال: List-Search$(L, ۴)$



x از گره نخست شروع می‌کند.

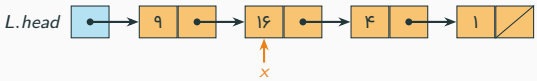
جست‌وجو در لیست: LIST-SEARCH

LIST-SEARCH(L, k)

1 $x = L.head$
2 **while** $x \neq$ NIL **and** $x.key \neq k$
3 $\quad x = x.next$
4 **return** x

CLRS, 4th ed.، صفحه ۲۶۰

مثال: LIST-SEARCH$(L, ۴)$



$۴ \neq ۹$، پس x یک گره جلو می‌رود.

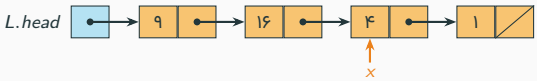
جست‌وجو در لیست: LIST-SEARCH

LIST-SEARCH(L, k)

```
1  x = L.head
2  while x ≠ NIL and x.key ≠ k
3      x = x.next
4  return x
```

صفحه ۲۶۰، *CLRS, 4th ed.*

مثال: LIST-SEARCH$(L, ۴)$



$۴ \neq ۱۶$، پس باز یک گره جلو می‌رود.

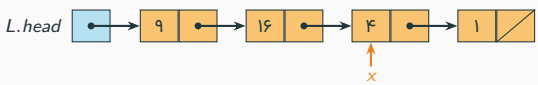
جست‌وجو در لیست: LIST-SEARCH

LIST-SEARCH(L, k)

```
1  x = L.head
2  while x ≠ NIL and x.key ≠ k
3      x = x.next
4  return x
```

CLRS, 4th ed.، صفحه ۲۶۰

مثال: LIST-SEARCH$(L, ۴)$



$x.key = ۴$، پس حلقه تمام می‌شود و اشاره‌گر به گره سوم برمی‌گردد. برای $k = ۷$ حلقه از همه‌ی گره‌ها می‌گذرد و nil برمی‌گردد.

زمان جست‌وجو

- هر دور حلقه یک گره جلو می‌رود و زمان ثابت دارد.
- بدترین حالت: کلید در لیست نیست یا در گره آخر است. حلقه از همه‌ی n گره می‌گذرد، پس زمان $\Theta(n)$ است.

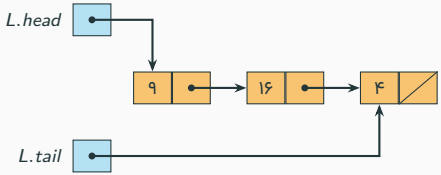
اشاره‌گر انتها

درج در انتهای لیست یک‌سویه



- لیست فقط $L.head$ را می‌شناسد. برای رسیدن به گره آخر باید همه‌ی گره‌ها را پیمود.
- پس درج در انتها $\Theta(n)$ است، هرچند خود درج فقط یک اشاره‌گر را تغییر می‌دهد.

اشاره‌گر انتها: $L.tail$



لیست، علاوه بر $L.head$، اشاره‌گر $L.tail$ به گره آخر را هم نگه می‌دارد.

```
INSERT-LAST(L, x)
1  n = ALLOCATE-NODE(x, NIL)
2  if ISEMPTY(L)
3     then L.head = n
4     else L.tail.next = n
5  L.tail = n
6  L.size = L.size + 1
```

هیچ حلقه‌ای نیست، پس INSERT-LAST از $O(۱)$ است.

اثر $L.tail$ روی رویه‌های قبل

```
DELETE-FIRST(L)
1  if ISEMPTY(L)
2     then error list is empty
3  n = FIRST(L)
4  L.head = n.next
5  FREE-NODE(n)
6  L.size = L.size − 1
7  if L.size == 0
8     then L.tail = NIL
```

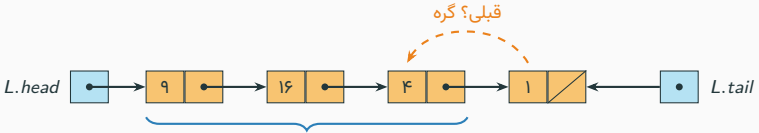
```
INSERT-FIRST(L, x)
1  L.head = ALLOCATE-NODE(x, L.head)
2  if L.size == 0
3     then L.tail = L.head
4  L.size = L.size + 1
```

- وقتی لیست از تهی به یک‌عنصری یا از یک‌عنصری به تهی می‌رود، $L.tail$ هم باید عوض شود.

رویه‌های کتاب دکتر قدسی، صفحه ۱۳۵، با نمادگذاری $CLRS$ و خط‌های افزوده‌ی نارنجی

حذف از انتها

پس از حذف گره آخر، $L.tail$ باید به گره ماقبل آخر اشاره کند. اما در لیست یک‌سویه هیچ گره‌ای نشانی گره قبلی خود را ندارد.



- $L.tail$ فقط گره آخر را نشان می‌دهد، نه گره پیش از آن.
- پس در لیست یک‌سویه، حذف از انتها حتی با $L.tail$ هم $\Theta(n)$ است.
- راه حل: هر گره، علاوه بر گره بعدی، گره قبلی را هم بشناسد.

لیست پیوندی دوسویه

لیست پیوندی دوسویه

«همان‌طور که شکل ۴.۱۰ نشان می‌دهد، هر عنصر یک لیست پیوندی دوسویه (doubly linked list) L شیئی است با یک صفت *key* و دو صفت اشاره‌گر: *next* و *prev*.»

CLRS, 4th ed.، صفحه ۲۵۸ (ترجمه ماشینی)

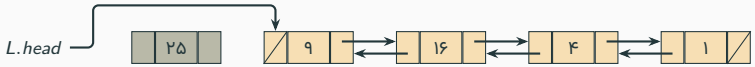


CLRS, 4th ed.، صفحه ۲۵۹ (بازترسیم شکل ۴.۱۰ الف)

- اگر $x.prev = \text{nil}$، گره x گره نخست (head) است. اگر $x.next = \text{nil}$، گره x گره آخر (tail) است.
- $L.head$ به گره نخست اشاره می‌کند. اگر $L.head = \text{nil}$، لیست تهی است.

درج در ابتدا: List-Prepend

```
List-Prepend(L, x)
1  x.next = L.head
2  x.prev = NIL
3  if L.head ≠ NIL
4      L.head.prev = x
5  L.head = x
```

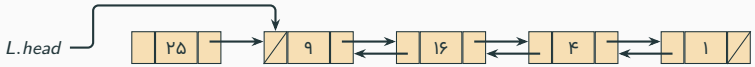


گره x ساخته شده ولی هنوز به لیست وصل نیست.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۰ و بازترسیم شکل ۴.۱۰ ب از صفحه ۲۵۹

درج در ابتدا: List-Prepend

```
List-Prepend(L, x)
1  x.next = L.head
2  x.prev = NIL
3  if L.head ≠ NIL
4      L.head.prev = x
5  L.head = x
```

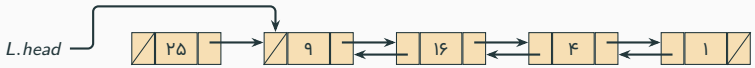


خط ۱: $x.next$ به گره نخست کنونی (۹) اشاره می‌کند.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۰ و بازترسیم شکل ۴.۱۰ ب از صفحه ۲۵۹

درج در ابتدا: List-Prepend

```
List-Prepend(L, x)
1  x.next = L.head
2  x.prev = NIL
3  if L.head ≠ NIL
4      L.head.prev = x
5  L.head = x
```

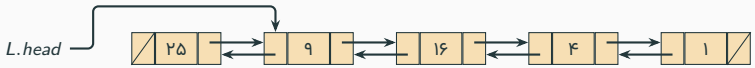


خط ۲: x گره نخست می‌شود، پس گره قبلی ندارد.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۰ و بازترسیم شکل ۴.۱۰ ب از صفحه ۲۵۹

درج در ابتدا: List-Prepend

```
List-Prepend(L, x)
1  x.next = L.head
2  x.prev = NIL
3  if L.head ≠ NIL
4      L.head.prev = x
5  L.head = x
```



خط‌های ۳ و ۴: لیست تهی نیست، پس *prev* گره ۹ به *x* اشاره می‌کند.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۰ و بازترسیم شکل ۴.۱۰ ب از صفحه ۲۵۹

درج در ابتدا: List-Prepend

```
List-Prepend(L, x)
1  x.next = L.head
2  x.prev = NIL
3  if L.head ≠ NIL
4      L.head.prev = x
5  L.head = x
```



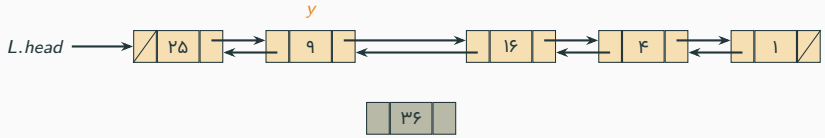
خط ۵: $L.head$ به x اشاره می‌کند. هیچ حلقه‌ای نبود، پس زمان $O(1)$ است.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۰ و بازترسیم شکل ۴.۱۰ ب از صفحه ۲۵۹

درج پس از یک گره: LIST-INSERT

x با $x.key = ۳۶$ پس از گره y (کلید ۹) درج می‌شود. رویه به خود لیست L نیازی ندارد، چون فقط اشاره‌گرهای گره‌ها را عوض می‌کند.

```
LIST-INSERT(x, y)
1  x.next = y.next
2  x.prev = y
3  if y.next ≠ NIL
4      y.next.prev = x
5  y.next = x
```



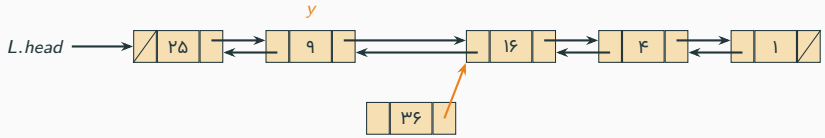
گره x ساخته شده و y گره ۹ است.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ پ از صفحه ۲۵۹

درج پس از یک گره: LIST-INSERT

x با $x.key = ۳۶$ پس از گره y (کلید ۹) درج می‌شود. رویه به خود لیست L نیازی ندارد، چون فقط اشاره‌گرهای گره‌ها را عوض می‌کند.

```
LIST-INSERT(x, y)
1  x.next = y.next
2  x.prev = y
3  if y.next ≠ NIL
4      y.next.prev = x
5  y.next = x
```



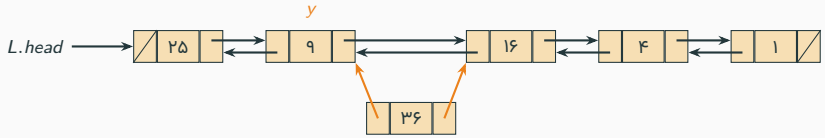
خط ۱: $x.next$ به گره پس از y (۱۶) اشاره می‌کند.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ پ از صفحه ۲۵۹

درج پس از یک گره: LIST-INSERT

x با $x.key = ۳۶$ پس از گره y (کلید ۹) درج می‌شود. رویه به خود لیست L نیازی ندارد، چون فقط اشاره‌گرهای گره‌ها را عوض می‌کند.

```
LIST-INSERT(x, y)
1  x.next = y.next
2  x.prev = y
3  if y.next ≠ NIL
4      y.next.prev = x
5  y.next = x
```



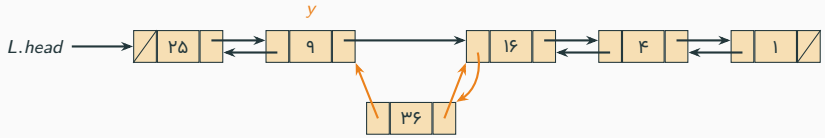
خط ۲: $x.prev$ به y اشاره می‌کند.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ پ از صفحه ۲۵۹

درج پس از یک گره: LIST-INSERT

x با $x.key = ۳۶$ پس از گره y (کلید ۹) درج می‌شود. رویه به خود لیست L نیازی ندارد، چون فقط اشاره‌گرهای گره‌ها را عوض می‌کند.

```
LIST-INSERT(x, y)
1  x.next = y.next
2  x.prev = y
3  if y.next ≠ NIL
4      y.next.prev = x
5  y.next = x
```



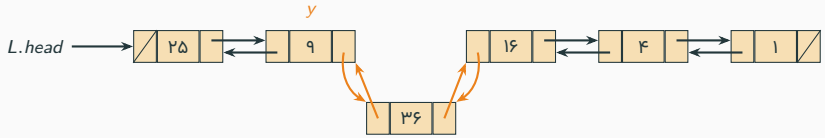
خط‌های ۳ و ۴: *prev* گره ۱۶ از y به x تغییر می‌کند.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ پ از صفحه ۲۵۹

درج پس از یک گره: LIST-INSERT

x با $x.key = ۳۶$ پس از گره y (کلید ۹) درج می‌شود. رویه به خود لیست L نیازی ندارد، چون فقط اشاره‌گرهای گره‌ها را عوض می‌کند.

```
LIST-INSERT(x, y)
1  x.next = y.next
2  x.prev = y
3  if y.next ≠ NIL
4      y.next.prev = x
5  y.next = x
```



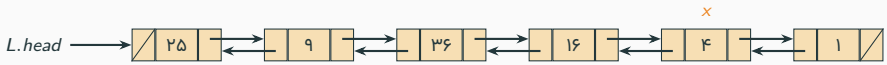
خط ۵: $y.next$ به x اشاره می‌کند. لیست اکنون $\langle ۲۵, ۹, ۳۶, ۱۶, ۴, ۱ \rangle$ است و زمان $O(۱)$ است.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ پ از صفحه ۲۵۹

حذف یک گره: List-Delete

x گره با کلید ۴ است. رویه باید خود اشاره‌گر x را بگیرد. برای حذف با کلید، اول List-Search را صدا می‌زنیم.

```
List-Delete(L, x)
1  if x.prev ≠ NIL
2      x.prev.next = x.next
3  else L.head = x.next
4  if x.next ≠ NIL
5      x.next.prev = x.prev
```



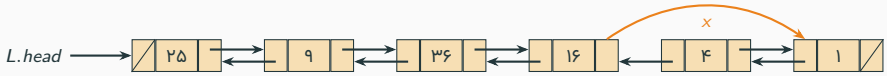
می‌خواهیم گره x را از لیست بیرون بیاوریم.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ ت از صفحه ۲۵۹

حذف یک گره: List-Delete

x گره با کلید ۴ است. رویه باید خود اشاره‌گر x را بگیرد. برای حذف با کلید، اول List-Search را صدا می‌زنیم.

```
List-Delete(L, x)
1  if x.prev ≠ NIL
2      x.prev.next = x.next
3  else L.head = x.next
4  if x.next ≠ NIL
5      x.next.prev = x.prev
```



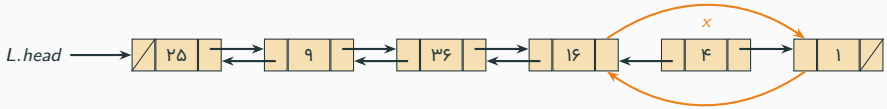
خط‌های ۱ و ۲: x گره نخست نیست، پس $next$ گره قبلی (۱۶) از روی x می‌پرد.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ ت از صفحه ۲۵۹

حذف یک گره: List-Delete

x گره با کلید ۴ است. رویه باید خود اشاره‌گر x را بگیرد. برای حذف با کلید، اول List-Search را صدا می‌زنیم.

```
LIST-DELETE(L, x)
1  if x.prev ≠ NIL
2      x.prev.next = x.next
3  else L.head = x.next
4  if x.next ≠ NIL
5      x.next.prev = x.prev
```



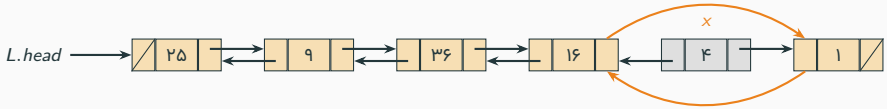
خط‌های ۴ و ۵: x گره آخر نیست، پس $prev$ گره بعدی (۱) هم از روی x می‌پرد.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ ت از صفحه ۲۵۹

حذف یک گره: List-Delete

x گره با کلید ۴ است. رویه باید خود اشاره‌گر x را بگیرد. برای حذف با کلید، اول List-Search را صدا می‌زنیم.

```
List-Delete(L, x)
1  if x.prev ≠ NIL
2      x.prev.next = x.next
3  else L.head = x.next
4  if x.next ≠ NIL
5      x.next.prev = x.prev
```



x از زنجیره بیرون افتاده است. زمان $O(1)$ است، چون گره قبلی را از $x.prev$ داریم.

CLRS, 4th ed.، شبه‌کد صفحه ۲۶۱ و بازترسیم شکل ۴.۱۰ ت از صفحه ۲۵۹

هزینه حذف در لیست دوسویه

لیست یک‌سویه

برای حذف گره x باید $next$ گره قبلی را عوض کنیم. گره قبلی را فقط با پیمایش از ابتدا پیدا می‌کنیم، پس حذف $\Theta(n)$ است.

لیست دوسویه

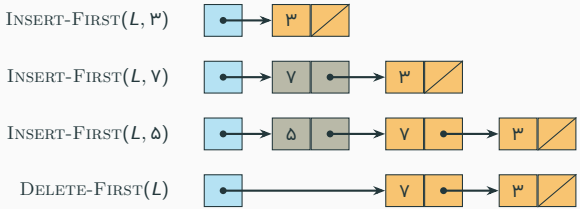
گره قبلی همان $x.prev$ است، پس LIST-DELETE از $O(1)$ است.

- حذف با کلید هنوز $\Theta(n)$ است، چون اول باید گره را با LIST-SEARCH پیدا کنیم.
- بهای $O(1)$ شدن حذف: یک اشاره‌گر اضافه در هر گره و کار بیشتر در هر درج، چون $prev$ هم باید به‌روز شود.

مقدمه‌ای بر پشته و صف

اگر فقط در ابتدا درج و حذف کنیم

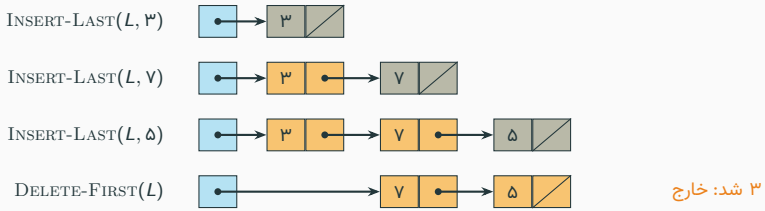
فقط Insert-First و Delete-First، هر دو $O(۱)$:



عنصری که آخر درج شد، اول خارج می‌شود. خانه‌ی آبی، $L.head$ است.

اگر در انتها درج و از ابتدا حذف کنیم

فقط Insert-Last (با $L.tail$) و Delete-First، هر دو $O(1)$:



عناصر به همان ترتیبی که درج شدند خارج می‌شوند. ($L.tail$ برای سادگی رسم نشده است.)

پشته و صف

«بر اساس مکان درج یا حذف عناصر، داده‌ساختار لیست به اسامی زیر شناخته می‌شود:

- پشته: درج و حذف فقط در یک انتهای لیست انجام می‌شود. در این‌صورت، اولین عنصری که از آن خارج می‌شود، آخرین عنصر درج شده است، یا اولین عنصر درج شده، آخرین عنصری خواهد بود که از آن خارج می‌شود.
- صف: درج فقط در انتها و حذف فقط از ابتدای لیست امکان‌پذیر است. در نتیجه، عناصر به همان ترتیبی که در صف درج شده‌اند از آن نیز خارج می‌شوند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۱۳۲

پشته: INSERT-FIRST و DELETE-FIRST

صف: INSERT-LAST و DELETE-FIRST

منابع

منابع

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition, MIT Press, 2022
- محمد قدسی – داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷